Packet Sniffing: A Brief Introduction

Sabeel Ansari, Rajeev S.G. and Chandrashekar H.S.

with tips on how to get the Network Interface card into a promiscuous mode

Packet sniffing is a method of tapping each packet as it flows across the network; i.e., it is a technique in which a user sniffs data belonging to other users of the network. Packet sniffers can operate as an administrative tool or for malicious purposes. It depends on the user's intent. Network administrators use them for monitoring and validating network traffic.

Packet sniffers are basically applications. They are programs used to read packets that travel across the network layer of the Transmission Control Protocol/Internet Protocol (TCP/IP) layer. (Basically, the packets are retrieved from the network layer and the data is interpreted.)

Figure 1 shows a typical packet sniffer program running on Ethernet. The packet sniffer listens to the data that arrives at the Network Interface Card (NIC). However, packet sniffers are not limited to Local Area Networks (LANs). Similar packet sniffers exist for Wide Area Networks (WANs). If a machine is in the path of two connected machines (A and B) on a WAN, the machine can listen to the traffic flowing from A to B.

An analogy to a packet sniffer is a telephone wiretap. A person can tap a telephone line if he or she wants to snoop on another person. Similarly, packet sniffers can be used to snoop on other people's data that is currently being transmitted across the network. Network sniffers can capture passwords and other sensitive pieces of information passing through the network.

Sometimes, sniffers can retain anonymity if they are launched from another system on the network. The packets may be sniffed from some intermediary hosts to which the launcher has access.

Sniffing programs can be classified under two categories. Commercial packet sniffers used by network administrators to help maintain networks, and underground packet sniffers used by those folks who sniff sensitive information for personal gain.

Typical uses of such sniffing programs include:

- Logging network traffic.
- Solving communication problems such as: finding out why computer A cannot communicate with computer B. (e.g. The communication may not be possible because of various reasons, such as a problem in either the system or the transmission medium.)
- Analyzing network performance. This way the bottlenecks present in the network can be discovered, or the part of the network where data is lost (due to network congestion) can be found.
- Retrieving user-names and passwords of people logging onto the network.
- Detecting network intruders.

There are a lot of packet sniffers available on the Internet. The techniques used to sniff packets on a shared bus broadcast network are different from the techniques used to sniff data on switched networks.

The broadcast sniffing technique

The technique behind packet sniffing on shared bus broadcast LANs is explained with the following example. IEEE 802.3 Ethernet LANs employ a broadcast technology, i.e. when a message is sent to another machine on the LAN, the message is sent to all the machines that are connected to the network. The machine's Network Interface Card (NIC) checks the destination address of the arriving packet. The card accepts the packet if it has the latter machine's address; otherwise, it is discarded.

What a packet sniffer does is put the NIC into a "promiscuous mode." The NIC now does not discard packets that are not addressed to its machine. It silently accepts the packets.

Before we explain how the NIC is put into a promiscuous mode, let's take a look at a few things about the Ethernet Card. When a machine communicates with another machine on Ethernet media, it specifies the destination machine's Internet Protocol (IP) address and the port number. The IP address (a 32-bit address) specified by the process is converted to that particular card's Media Access Control (MAC) address (a 48-bit low-level address). The packets are then inserted into MAC frames (Fig. 2) for transmission.

Each IP packet is split so that the packets fit into the MAC frames, which contain the destination address. The MAC frames are usually up to 1500 bytes. Note that MAC addresses are unique. Each Ethernet card manufactured contains a unique address that is hard coded into the card. MAC addresses are made up of 48 bits with the first 24 bits specifying the vendor. So all cards manufactured by a specific company will have the same first 24 bits in all their NICs. The next 22 bits specify the serial number of the card, which is assigned by the vendor. One bit of the remaining two bits indicates whether it is a broadcast/multicast address. The other bit indicates if the adapter has been assigned some address locally (the system administrator may reassign some MAC addresses).

Steps for creating a Linux packet sniffer

The main steps in the development of a packet sniffer are:

1) Creating a socket stream.

2) Setting the NIC into a promiscuous mode.

3) Reading data from the open socket stream.

The rest of the steps only deal with interpreting the headers and formatting the data (and redirecting the data to the output stream).

Creating a socket. On UNIX and its clones, communication points called *sockets* can be created. These help in the communication of end systems present in the network. When a socket is created, a socket stream, similar to the file stream, is created, through which data is read.

Setting the NIC to promiscuous mode. First, a reference to a structure called *ifreq* is needed. This is done by

 0278-6648/02/$17.00 © 2002 IEEE

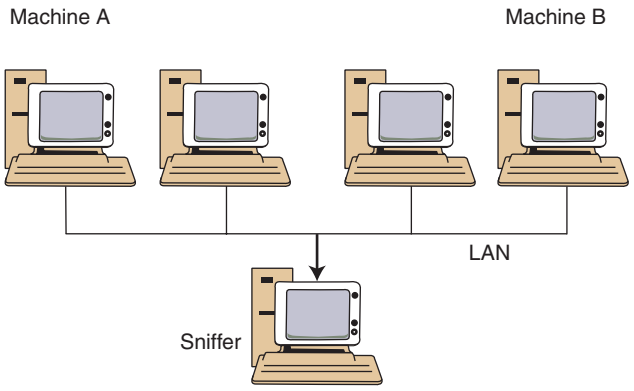

Fig. 1 Local Area Network

the statement, *struct ifreq ifr*. The *ifreq* structure is a rather large one that is passed on to the standard *ioctl*'s to configure the network devices.

The *ifreq* is an interface request structure used for socket *ioctl*. (The *ioctl* provides an interface for controlling the behavior of devices, their descriptors and configuring the underlying services.)

Next, the NIC name, viz. eth0 (usually), is copied to the structure member *ifr_name*.

The next step is to get the flags of the specified interface using the *ioctl* system call. The *ioctl* system call takes three arguments, viz.

1) The socket stream descriptor

2) The function that the *ioctl* function is supposed to perform. Here, the macro used is SIOCGIFFLAGS.

3) Reference to the *ifreq* member

In the next step, the *promisc* flag must be set. (The *promisc* flag is a structure element of the *ifreq* structure. Setting this flag makes the NIC accept all the packets on the network. The IFF_PROMISC is a predefined macro.) It is set using the statement,

Ifr.ifr_flags |= IFF_PROMISC;

The last step in this process is to set these flags to the NIC using the *ioctl* call. The same first and third parameters are used, but the second parameter to *ioctl* is changed to SIOCSIFFLAGS.

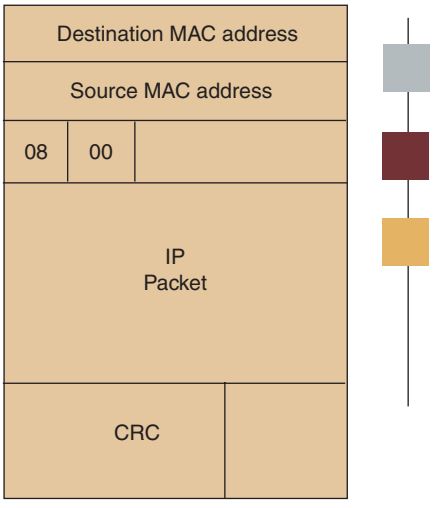

Fig. 2 MAC frame format

Now comes the cumbersome part: protocol interpretation. To do this, the user is required to have some basic knowledge about the protocol that he or she intends to sniff. The protocol contents, its fields, field lengths must be known and the user must know what kind of data to expect in those fields.

To interpret the headers, the fields of the protocol must be accessible. On a Linux machine, by including the headers <linux/ip.h> and <linux/tcp.h>, the IP and TCP protocols can be interpreted. These headers contain structures that represent the IP and TCP headers. The user can refer to these headers and interpret the information that is passing across the network. A more complete sniffer would not just be able to capture a single protocol, but all types of packets.

For example, let's look at a protocol interpretation at a File Transfer Protocol (FTP) running on the TCP/IP stack (which in turn is run on IEEE 802.3 Ethernet LAN). The Ethernet header is 14 bytes in length, followed by the IP header—which is 20 bytes and the TCP header—which is also 20 bytes. If we remove these headers from the set of bits we have in hand, i.e., one frame, we will be left with the data packet of the application (here, FTP). Now the user can discern the information being transferred between the application peers.

Countermeasures

A malicious hacker trying to find a way around security measures is a fairly frequent occurrence, if not the norm. Likewise, methods have been developed to thwart these types in the field of packet sniffing. The user can employ a number of techniques to detect sniffers on a network. There are even methods that can be employed to stop these sniffers from functioning. Some of the techniques are listed here.

Sniffer detection

There are quite a few methods to detect sniffers on the network.

• The user can generate packets that contain an invalid address. If a machine (on the network) accepts the packets, it can be concluded that the machine is running a sniffer. Here is one method to do it:

The user can change the MAC address of his or her machine temporarily. Sending packets to the old address of the machine should not result in the acceptance of that packet. If a machine does accept the packet, then it is running a sniffer.

• Software programs such as *AntiSniff* can be used to detect sniffers. According to a review by Dave Kearns (*Network world on NT*), "*AntiSniff* gives…the ability to remotely detect computers that are packet sniffing. By running a number of non-intrusive tests in a variety of ways network administrators and information security professionals can determine whether or not a remote computer is listening in on all network communications." (Note: *AntiSniff* is commercial software. See the web site listed in *Read more about it*.)

• Simple Network Management Protocol (SNMP) monitoring can be used. SNMP helps network managers locate and correct problems on the TCP/IP internet. Managers invoke an SNMP client on the local computer, and use the client to contact one or more SNMP servers. SNMP uses a fetch and store model in which each server (there may exist more than one) maintains a set of conceptual variables that include statistics, such as count of packets received, etc. Using SNMP, one can log connections and disconnections to the ports; thus, it reveals the existence of a sniffer in the network.

Protection from packet sniffers

The user can employ a number of techniques to protect his or her data:

• A packet switched network can be set up. This will make the previously discussed type of sniffers useless. How? In packet switched networks, each station is connected to a hub. Each line card in the hub has a buffer to hold an incoming frame. A fast backplane transfers the packets from one line card to another. The latter line card then transmits the packets to the station to which it is connected. Thus, a sniffer running on one machine will not be able to "listen" to other transfers in the network.

• Sensitive data can be encrypted

before being transmitted across the network. While this won't prevent a sniffer from functioning, it will ensure that the sniffer reads junk and, hopefully, is not able to decipher it.

Conclusion

Packet sniffers are utilities that can be efficiently used for network administration. At the same time, it can also be used for nefarious activities. However, a user can employ a number of techniques to detect sniffers on the network and protect the data from sniffers.

Acknowledgement

We are thankful to Mr. Anil Kumar K M, Lecturer, Department of Computer Science & Engineering, Vidya Vardhaka College of Engineering, Mysore, India for his valuable guidance.

Read more about it

• Comer, Douglas E., *Internetworking with TCP/IP: Volume 1 – Principles, Protocols and Architecture*, Prentice Hall India, 3rd edition © 2000.

• Stevens, Richard, *TCP/IP Illustrated: Volume 1*, Pearson Education Asia, © 2001.

• Stevens, Richard, *UNIX Network Programming*, Prentice Hall India, © 2001.

• Stevens, Richard, *Advanced programming in the UNIX environment*, Addison-Wesley Professional Computing Series © 2001.

• Stones, Richard & Matthew, Niel, *Beginning Linux Programming*, Wrox Publishers, © 1999.

• Sniffing FAQ <http://www.robert-graham.com>

• Sniffer resources <http://packet-storm.decepticons.org>

• AntiSniff <http://packetstormsecurity.nl/sniffers/antisniff/>

About the authors

Sabeel Ansari <sabeel_ansari@yahoo.com>, Rajeev S. G. <rajeev_sg@yahoo.com> and Chandrashekar H. S. <chandrashekar_hs@yahoo.com> are final year students of the Computer Science & Engineering discipline. Sabeel Ansari is an IEEE Computer Society and IEEE Communications Society student member.

They are studying to get their Bachelor of Engineering degree from Visveswaraiah Technological University, Belgaum at Vidya Vardhaka College of Engineering, Mysore, India.